RC1

1. Basic Linux command

1.1 `cd`

```
cd <pathname>
```

go to the directory `<pathname>`.

Special characters for directories:

- root directory: `/`
- home directory: `~`
- current directory: `.`
- parent directory: `..`

1.2 `ls`

```
ls <directory>
```

list the `<directory>`

Argument options:

- `-l` : list in long format
- `-a` : list all files including the hidden files

```
-rw-rw-r-- 1 EggHead_o EggHead_g 40 Feb 23 10:15 LongFormat.txt
```

Long Format of File Information

- `-rw-rw-r--` : permission
  - First character: `-` regular file; `d` directory
  - Next three: read, write, execution permission of the owner
  - Next three: read, write, execution permission of the group

    - Final three: read, write, execution permission of everyone else
- `EggHead_o` : owner
- `EggHead_g` : group
- `40` : file size in bytes
- `Feb 23 10:15` : modification time
- `LongFormat.txt` : file name

1.3 `mkdir`

```
mkdir <pathname/directory>
```

Create `<directory>` under `<pathname>`

1.4 `rmdir`

Remove `<directory>` under `<pathname>`, `<directory>` should be empty

```
rmdir <pathname/directory>
```

1.5 `touch`

```
touch <pathname/filename>
```

Create an empty `<filename>` under `<pathname>`

1.6 `cp`

```
cp <source> <dest>
```

- `cp <file1> <file2>` : copy the content of `<file1>` into `<file2>`
- `cp <filelist> <directory>` : copy files in `<filelist>` into `<directory>`
  - `cp <file1> <file2> ... <fileN> <directory>` : copy multiple files into `<directory>`
  - `cp <ST>*<ED> <directory>` : copy any file starts with `<ST>` and ends with `<ED>` into `<directory>`
    - `*` : wildcard, can represent any character string (even empty)
- `cp -r <directory1> <directory2>` :
  - If `<directory2>` does not exist → copy as `<directory2>`.
  - If `<directory2>` exists → copy inside `<directory2>`

1.7 `mv`

```
mv <source> <dest>
```

- `mv <file1> <file2>` : rename
- `mv <file> <directory>` : move file
- `mv <directory1> <directory2>`
  - If `<directory2>` does not exist → rename
  - If exists → move inside

Argument options:

- `-i` : prompt before every removal

1.8 Edit `filename`

```
nano <filename>
gedit <filename>
vim <filename>
```

1.9 Show `filename`

```
cat <filename>
less <filename>
```

1.10 I/O Redirection

- `>` : redirect standard output to a file
- `<` : redirect standard input from a file

1.11 `diff`

```
diff <file1> <file2>
```

- If same → no output
- If different:
  - `<` → from `<file1>`
  - `>` → from `<file2>`

Summary line:

- `c` : change
- `a` : add
- `d` : delete

Argument options:

- `-w` : ignore white spaces

---

2. Developing and Compiling Programs on Linux

2.1 Compile the program

```
g++ -o <source.cpp>
```

Argument options:

- `-o` : output file name
- `-g` : debugging info
- `-Wall` : enable warnings

2.2 Run the program

```
<pathname>/<programName>
```

2.3 Compilation process

```
source code (.cpp) ---> Object code (.o) ---> executable program
```

compile: `g++ -c <source.cpp>`
link: `g++ -o <program name> <source.o>`

2.4 Developing Program on Linux

Multiple Source Files

- Header files (`.h`)
  → function declarations & class definitions
- Source files (`.cpp`)
  → function implementations

Example:

```cpp
// add.h
#ifndef ADD_H
#define ADD_H
int add(int a, int b);
#endif
```

```cpp
// add.cpp
int add(int a, int b) {
    return a + b;
}
```

```cpp
// run_add.cpp
#include "add.h"

int main() {
    add(2, 3);
    return 0;
}
```

Header Guards

The process of `#include` is replacing the `#include` line with the content of the header file. If we include the same header file multiple times, the compiler will complain about the redefinition of the classes and functions. The header guards are used to prevent this from happening.

```cpp
#ifndef VAR
#define VAR
...
#endif
```

2.5 Makefile

For large projects, it's not convenient to compile all the files manually. We can use process. make to automate the make is a build automation tool that automatically builds executable programs and libraries from source code by reading files called `Makefile` which specify how to derive the target program.

Basic syntax:

```
Target: Dependencies
    Command
```

Example:

```
all: my_executable

my_executable: my_program.o my_class.o
    g++ -std=c++17 -Wall -g -o my_executable my_program.o my_class.o
```

```
my_program.o: my_program.cpp
    g++ -std=c++17 -Wall -g -c my_program.cpp

my_class.o: my_class.cpp
    g++ -std=c++17 -Wall -g -c my_class.cpp

clean:
    rm -f my_executable *.o
```

Makefile (with variables)

```
CXX = g++
CXXFLAGS = -std=c++17 -Wall -g

all: my_executable

my_executable: my_program.o my_class.o
    $(CXX) $(CXXFLAGS) -o my_executable my_program.o my_class.o

my_program.o: my_program.cpp
    $(CXX) $(CXXFLAGS) -c my_program.cpp

my_class.o: my_class.cpp
    $(CXX) $(CXXFLAGS) -c my_class.cpp

clean:
    rm -f my_executable *.o
```

---

3. Review of C++ Basics

3.1 lvalue and rvalue

- lvalue: An expression which may appear as either the left-hand or right-hand side of an assignment.
  - any non-constant variable is an lvalue
- rvalue: An expression which may appear on the right- but not left-hand side of an assignment.
  - any constant is an rvalue

Examples:

```
int a = 1;          // a: lvalue, 1: rvalue
int b = a;          // b: lvalue, a: lvalue
const int c = a + b; // c is an rvalue, a + b is an rvalue
int *p = &a;        // p is an lvalue, &a is an rvalue
```

3.2 Function Declaration and Definition

Declaration:

```
// return_type function_name(parameter_list);
int add(int a, int b);
void print(string s);
```

Note that this should come before the function is called.

Definition:

```
/*
return_type function_name(parameter_list) {
    function_body;
}
*/

int add(int a, int b) {
    return a + b;
}

void print(string s) {
    cout << s << endl;
}
```

3.3 Pointers, References, and Arrays

Pointers: a variable that stores the address of another variable. Changing the value of a pointer will change the value of the variable it points to.

```
int a = 1;
int *p = &a; // p is a pointer to a, &a is the address of a
cout << *p << endl; // *p is the value of a
*p = 2; // a is now 2
```

- `*` : the dereference operator, which returns the value of the variable that the pointer points to.
- `&` : the address-of operator, which returns the address of a variable.

Some special cases:

```
int* p1, p2; // Here p2 is not a pointer. Where declaring, '*' only acts to
the variable next to it.

int *q1, *q2; // Here q1 and q2 are pointers.
q1 = nullptr; // When initializing pointers, please give it a value.

int c = 0;
p1 = &c;
cout << p1; // The direct value of p1 is the address of c, like 0x7fffc000.
It is
            // unsure and depends on you system and run-time environment.

int a = 1;
int b = 2;

int *p = &a; // int* p = &a is also ok!
int** pp = &p; // pp is a pointer to p, which means the type of pp is int**

*pp = &b; // *pp is what pp points to
*p = a; // *p is what p points to
**pp = b; // **pp is *(*pp)

*&b = 6; // This is weird but legal. *&b is *(&b), where &b is the address
of b,
         // and * operator on (&b) returns to b itself. But &*b is invalid.
Why?

int **qq = &(&b) // This is WRONG! Remember that &b is an rvalue and &
operator
                 // can only act on lvalues!

void *v = nullptr; // v is a void pointer. A void pointer can point to
anything.
                   // But we cannot directly use *v. We should use
static_cast.
int s1 = 0;
double s2 = 0;
char s3 = '0';

v = &s1; // legal
v = &s2; // legal
v = qq; // legal
v = &s3; // legal
```

```
p = &s3; // illegal. p is a pointer to int but s2 is char.
cout << *v; // illegal

char *pc = static_cast<char*>(v); // legal. Turn v from void* to char*
cout << *pc; // should be '0'
```

References: an alias of a variable. Changing the value of a reference will change the value of the variable it refers to.

```
int a = 1;
int &r = a;// & is the reference operator, r is a reference to a
r = 2;// a is now 2
```

- A reference must be initialized when it is declared.
- After initialization, a reference cannot be changed to refer to another variable.

Arrays: a collection of variables of the same type. The size of an array must be known at compile time. You can access the elements of an array using `[]` operator or by pointers.

```
int a[5] = {1, 2, 3, 4, 5}; // a is an array of 5 integers
cout << a == &a[0] << endl; // true
cout << a[0] << endl; // 1
int b[5] = {1}; // This will make b[0] = 1, b[1] ~ 4 = 0
int c[5]; // This will make initial values uncertain unless it is defined as
global variable.
a = b; // illegal! a is an rvalue.
```

Arrays work well with pointers.

```
int a[5] = {0, 1, 2, 3, 4}; // The memory layout of an array is contiguous.
int *p = a; // This let p point to a[0].
// The calculation of pointers can be viewed as moving the pointer
"forwards" or "backwards". Please do this with arrays.
*(p + 1) = -1; // This is equal to a[1] = -1.
p = p + 2; // Now p points to a[2] and *p is 2.
p++; // Now p points to a[3] and *p is 3.
p--; // Now p points to a[2] and *p is 2.
p[2] = -2; // p[2] means *(p + 2)

// A fun thing: *(p + 2) is equal to *(2 + p), so...
2[p] = -2; // This is even legal.
```

3.4 Function Call Mechanism

When a function is called, the parameters are passed to the function. There are two ways to **Function Call Mechanism** pass parameters:

- **Pass by value**: the value of the parameter is copied to the function. Modifying the parameter inside the function will not affect the original variable.
- **Pass by reference**: the address of the parameter is passed to the function. Modifying the parameter inside the function will affect the original variable.
  - With **pointers**: You can directly change the value of the variable that the pointer points to. This avoid copying instances of large objects sometimes.
  - With **references**: More readable and intuitive than pointers.

In C++, arrays are passed by reference

```
void f(int a[]) {
    a[0] = 1; // passed by reference, the value of a[0] is changed
}
```

3.5 Structs

Structs are user-defined data types that can contain multiple variables of different types. The members of a struct can be common data types or other structs.

```
struct Student {
    string name;
    int id;
    double gpa;
};// don't forget the semicolon here
```

Usage:

```
Student s1;// s1 is a struct variable
s1.name = "Alice";
s1.id = 52337091;
s1.gpa = 4.0;

Student s2 = {"Bob", 52337092, 3.9};// another way to initialize

Student *s3 = &s1;// s3 is a pointer to s1
s3->name = "Bob";// access the members of struct pointer using ->
```

4. Const Qualifier

`const` means there will be no modifications.

- Note that `const` is a kind of declaration.

```
const int a = 5;
int const a = 5;// same
```

4.1 Const with references

- Reference must be initialized with an object
- Non-const reference must be initialized with an lvalue
- Const reference can be initialized with an rvalue

```
const int a = 10;
const int a; //illegal
const int& ref1 = a; //legal
const int& ref2 = 2; //legal
ref1 = 2; //illegal
```

```
const int cInt = 0;
int& ref = cInt; // illegal const
int& cref = cInt; //legal
```

When calling big structures, we can use const to optimize time:

```
 struct BigStruct{
 //many things...
};
void f1(BigStruct& T);
void f2(const BigStruct& T); // This is quicker when passing T. But T should
not be changed!
```

4.2 Const with pointers

A pointer stores the address of an object. Two candicates for the `const` qualifier: the pointer itself (the address), and the object it points to.

```
const int* ptr2Intc; // the integer cannot be modified
int const* ptr2cInt; // the integer cannot be modified
int* const cPtr2Int; // the address cannot be modified

// Think about the following:
// Which are illegal?

int a = 1, b = 2;
const int c = 3;
const int d = 4;
```

```
const int* p1 = &c;
const int* p2 = &a;
int* const p3 = &b;
int* const p4 = &d; // wrong
const int* const p5 = &d;
p1 = &a;
*p1 = 6; // wrong
p3 = &a;
*p3 = 7; // wrong
```